

RUPTURE

Rapport technique officiel — Préparation architecture pour la publication future

Portée	Architecture uniquement — aucun changement de gameplay, d'équilibrage ni d'expérience. La bêta manuelle V2 est toujours en cours et n'est pas affectée.
Date du rapport	13 septembre 2026
Dépôt	scuillerjack-oss/rupture-v0 (public, inchangé)
Branche	main
Commit de départ (V2 + rapport)	4327c3605cbcbcd0ef527e1164ee819b2e3996a0
Commit final de cette passe	36cc2e1a9edefb1622d56a9d33bdfa1634cce470
URL permanente (inchangée)	https://scuillerjack-oss.github.io/rupture-v0/
Hébergement	GitHub Pages, déployé via GitHub Actions

AVERTISSEMENT DE PORTÉE : ce rapport documente une préparation architecturale. Il ne remplace pas et ne modifie pas les rapports gameplay (RUPTURE_V2_Rapport_Technique_Officiel.pdf reste la référence pour l'équilibrage et les résultats de simulation, tous inchangés ici).

1. Objectif de cette passe

Pendant que la bêta manuelle de RUPTURE V2 est en cours, préparer l'architecture technique du projet pour sa future commercialisation (version gratuite/Premium, publicité, publication Android/Google Play) — sans implémenter aucune de ces fonctions maintenant, et sans qu'aucun changement de gameplay, d'équilibrage ou d'expérience ne soit visible sur la version actuellement en bêta.

2. Audit préalable (état trouvé avant modification)

Le commit de départ (4327c36, V2 + rapport technique) a été examiné avant toute modification.

Déjà correctement séparé

- `src/engine/` (territoires, équilibrage, état, simulation) ne dépend d'aucune API DOM, d'aucune notion de plateforme ni de `localStorage` — un noyau de jeu pur.
- `src/ui/` ne fait que du templating à partir de l'état reçu en paramètre.
- `src/save.js` est l'unique point de contact avec `localStorage`.
- `src/main.js` est la seule « colle » DOM + moteur + sauvegarde + service worker.

Dette identifiée (avec justification produit concrète)

- Vitesses (0/1/2/4) codées en dur dans `src/ui/hud.js` : aucun point d'indirection pour la piste déjà actée « certaines vitesses pourraient devenir Premium en commercial ». Sans correction, cette règle commerciale aurait fini par s'écrire directement dans le template HTML du HUD.
- Aucune interface pour de futurs services commerciaux (Premium, publicité, analytics) : chaque intégration future aurait dû improviser un point d'entrée a posteriori dans plusieurs fichiers à la fois.
- Sauvegarde à bascule brutale (`SAVE_VERSION` différent → sauvegarde perdue). Sûr (jamais d'état incohérent) mais inadapté à de vrais joueurs Android qui ne devraient pas perdre leur progression à chaque mise à jour.
- Aucune distinction dev/bêta/commercial nulle part dans le code.

Rien d'autre n'a été jugé nécessiter une modification : `balance.js`, `simulation.js`, `territories.js`, le rendu de carte, le CSS, le pipeline CI/CD, le service worker et le manifeste PWA sont restés strictement inchangés.

3. Modifications effectuées

Fichiers créés

- `src/config/runtime.js` — configuration centralisée (environnement, vitesses disponibles). Tant que l'environnement n'est pas 'commercial' (aucun build de ce type n'existe), toutes les vitesses restent disponibles quel que soit le statut Premium.
- `src/services/premium.js`, `ads.js`, `analytics.js`, `index.js` — interfaces découplant le moteur des futurs SDK commerciaux ; adaptateurs locaux/no-op uniquement, aucun fournisseur réel intégré.
- `src/engine/migrations.js` — registre de migrations de sauvegarde (vide pour l'instant), avec garde-fou contre toute chaîne cyclique ou mal configurée.
- `docs/ARCHITECTURE.md`, `docs/TRAJECTOIRE_COMMERCIALE.md` — audit détaillé et trajectoire produit documentée.
- `tests/migrations.test.js`, `tests/services.test.js`, `tests/hud.test.js` — 11 nouveaux tests.

Fichiers modifiés (câblage minimal, sans effet visible)

- `src/save.js` — `loadState()` tente une migration avant de rejeter une version différente ; ajout de `getPremiumFlag/setPremiumFlag` (indicateur local, non utilisé nulle part dans l'UI).
- `src/ui/hud.js` — les boutons de vitesse proviennent désormais de `getAvailableSpeeds(services)` au lieu d'être codés en dur.
- `src/ui/screens.js`, `src/main.js` — propagation du paramètre `services` jusqu'au HUD ; assemblage des services via `createServices()` ; les erreurs JS non interceptées sont routées vers `services.analytics.logError` (no-op).
- `README.md` — pointeur vers les deux nouveaux documents.

Aucune ligne de `src/engine/balance.js`, `simulation.js`, `territories.js`, `state.js` (hors chargement du nouveau module de migrations, sans effet sur son comportement), du CSS ou du pipeline CI n'a été modifiée.

Stratégie d'empaquetage Android recommandée (documentée, non implémentée)

Analyse comparative Capacitor / TWA (Trusted Web Activity) documentée dans `docs/TRAJECTOIRE_COMMERCIALE.md`. Capacitor est recommandé, précisément parce que la feuille de route inclut Premium (Play Billing) et de la publicité (AdMob), deux besoins bien mieux couverts par de vrais plugins natifs que par les API web encore inégales qu'utiliserait TWA pour les mêmes besoins. Aucun projet Android/Capacitor n'a été ajouté au dépôt dans cette passe.

4. Tests et non-régression

45/45 tests passent (34 hérités de la V2 + 11 nouveaux : 5 sur migrations.js, 4 sur services/* et la configuration, 2 sur le rendu des vitesses par hud.js).

Vérification	Résultat
Batterie de 162 simulations (9 stratégies × 18 origines)	Rejouée : résultats identiques à ceux du rapport V2, aux variations aléatoires préexistantes près (stratégie semi-aléatoire et fermetures de routes, non déterministes de longue date, indépendantes de cette passe). docs/v2-simulation-results.json non modifié.
Vitesses ×1/×2/×4	Rendu vérifié identique en navigateur (■ 1x 2x 4x, mêmes valeurs, même ordre) après le passage par getAvailableSpeeds(), verrouillé par un nouveau test automatisé.
Menu / Paramètres / Difficulté / Secteurs	Parcours E2E complet rejoué sur un build frais : identique à la V2, 0 erreur console.
Sauvegarde	Un ancien format toujours rejeté proprement quand aucune migration n'est enregistrée (test conservé à l'identique + nouveaux tests sur le mécanisme de migration lui-même).
PWA	Manifest, icônes, service worker revérifiés actifs et servis sans erreur.
Mobile	360px et 412px : aucun débordement, aucune cible tactile sous 40px, 0 erreur console.

5. Ce qui est préparé, différé, ou hors de portée technique

Préparé maintenant (dans le dépôt, testé)

- Configuration centralisée dev/bêta/commercial.
- Interfaces de services découplées du moteur : Premium, Publicité, Analytics/erreurs.
- Indicateur Premium local et point d'indirection pour restreindre des vitesses en configuration commerciale (inactif en bêta, testé pour les deux cas via une configuration injectée).
- Registre de migrations de sauvegarde, avec garde contre tout état incohérent.
- Point d'entrée pour une future remontée d'erreurs (no-op aujourd'hui).
- Documentation d'audit et de trajectoire commerciale.

Volontairement non implémenté

- Tout projet Android/Capacitor réel dans le dépôt.
- Toute intégration réelle de Google Play Billing, d'AdMob ou d'un fournisseur analytics.
- Toute interface joueur pour un achat Premium (aucun écran d'achat n'existe).
- Moteur audio/musique (au-delà des réglages déjà visibles-mais-désactivés depuis la V2).
- Un pipeline de build/CI séparé pour un flavor « commercial ».
- Politique de confidentialité, formulaire Data safety, fiche store Play Console.

Nécessitera une décision ou un compte externe (de votre côté)

- Choix définitif du contenu Premium et de son modèle économique.
- Choix d'un fournisseur de publicité (AdMob ou autre) et création du compte associé.
- Décision sur l'usage ou non d'un service d'analytics/erreurs, et lequel.
- Création du compte développeur Google Play (payant).
- Décision sur le nom, l'identité visuelle finale et la fiche store.
- Décision sur l'hébergement définitif (GitHub Pages embarqué vs domaine propre) si une architecture hybride avec contenu distant est un jour souhaitée.
- Rédaction de la politique de confidentialité et réponses au questionnaire de classification par âge.
- Décision sur l'ajout de musique/effets sonores (création ou licence).

PRÉPARÉ ≠ PUBLIABLE : cette passe rend RUPTURE prêt à recevoir, plus tard et sans reconstruction, les briques listées ci-dessus. Elle ne rend PAS RUPTURE publiable sur Google Play : aucun projet Android n'existe, aucun compte développeur n'est créé, aucune conformité (confidentialité, sécurité des données, classification) n'est traitée, et le gameplay lui-même est encore en bêta manuelle. Cette distinction est volontairement stricte.

6. État du déploiement — vérification systématique du commit réellement en production

- Un seul commit poussé sur main : 36cc2e1a9edefb1622d56a9d33bdfa1634cce470.
- Run GitHub Actions #34744856704 (run_number 10) : job « build » (npm ci, npm test — 45/45, npm run build) et job « deploy » (actions/deploy-pages@v4) tous deux **success**.

- Log serveur du job de déploiement : pages_build_version correspondant exactement à 36cc2e1, Reported success!, URL évaluée <https://scuillerjack-oss.github.io/rupture-v0/> — la même URL permanente que pour toutes les versions précédentes.
- git ls-remote origin main confirme que la branche distante pointe exactement sur 36cc2e1.

Conformément à la règle permanente : cette correspondance commit testé = commit déployé est vérifiée directement dans les logs serveur de GitHub Actions, comme pour chaque version précédente.

Limite de vérification inchangée depuis la V0 : cette session ne peut pas charger elle-même l'URL publique (politique réseau sortant de l'environnement d'exécution).